Automated QA for the Eclypse platform

A robot that uses the platform like a real user, checks that every action worked, never touches client data, and explains failures in plain English. Run it with `npm run qa`.

1. How it works, when it runs, what it checks

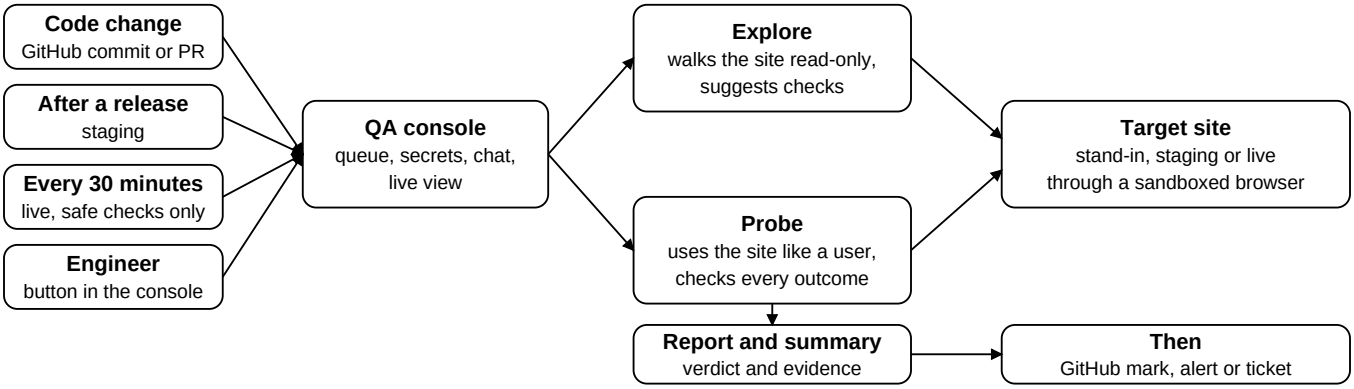

Figure 1. Any trigger goes through the QA console, which drives a sandboxed browser against the target and reports back.

When	Where	What runs
Every code change	Test build	Full check, AI mocked
After a release	Staging	Full check
Every 30 minutes	Live, fake company	Safe read-only checks

What it checks: results, not page loads.

- An upload shows up with the right row count and is still there after a reload.
- Delete removes that one document and nothing else.
- An AI job finishes and its numbers match the input file.
- Downloads match the screen. A button that does nothing is a failure.
- Wrong password, wrong code and a bad file each give a clear error.

2. Tools, and why

Choice	Why	Not chosen
Playwright	Real browser, waits on its own (less flaky), handles uploads and downloads, can block requests (the safety guard).	Cypress, Selenium
Find buttons by label	Clicks what a user sees, so a redesign does not break tests.	CSS selectors
Authenticator codes	Robot makes the same code a phone shows. No backdoor.	Email codes
Claude Haiku	Cheap and fast. Explores and plans once. Never decides pass or fail.	AI in the verdict
GitHub	Runs on every change and marks each commit pass or fail.	Manual runs

3. Signing in safely when sign-in needs a one-time code

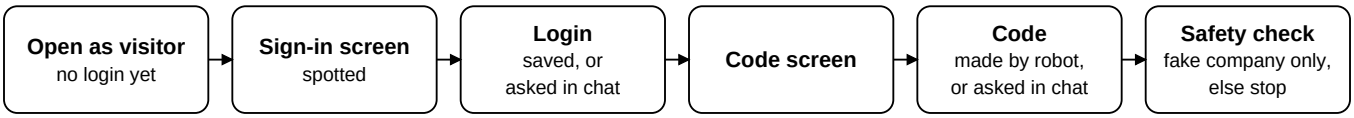

Figure 2. The robot signs in only when it reaches a sign-in screen, and stops if it lands anywhere but the synthetic company.

- **Real two-step sign-in.** The secret sits in a secret store. The robot makes the 6-digit code itself.
- **Fresh code every time.** It never reuses a code and never uses one that is about to expire.
- **One sign-in per run.** No overlapping runs, so no lockouts.
- **No saved login?** The run pauses and asks in the job chat.

4. Staying safe on the live platform

The rule: the robot can only see and change synthetic data, and it stops if it cannot prove that.

1. **Own company only.** The QA account belongs to a fake company. It has no way into client accounts.
2. **Stop if unsure.** After sign-in it checks where it landed and stops if it is not the fake company.
3. **Write guard.** Every save or delete is checked in the browser. It can only delete what it created.
4. **Canary.** A second fake company holds marked records. If that marker ever shows up, it is a critical failure.
5. **Clean evidence.** Passwords and codes are blacked out in screenshots and the live view.

5. AI output that changes on every run, and cost

Check facts, ignore wording. The test file has known answers. The AI's numbers must match them. The wording can change every run and is never pass or fail.

Long jobs. It fails early with a clear reason, like "stuck at 75% for 15 seconds", not a vague timeout.

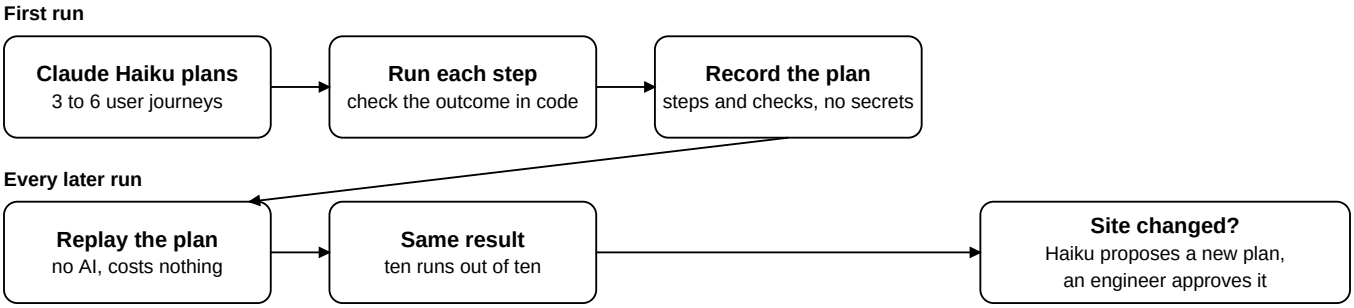

Figure 3. On a new site, Claude plans once. The recorded plan then replays without AI, so results repeat and cost nothing.

Where	AI	Cost
Every code change	Mocked	Free
Release	Real, recorded once, then replayed	Near free
Live checks	Real Haiku, smallest file	Under $1 a month
New site	Haiku with step and token caps	Cents per run

6. What happens when it finds a problem

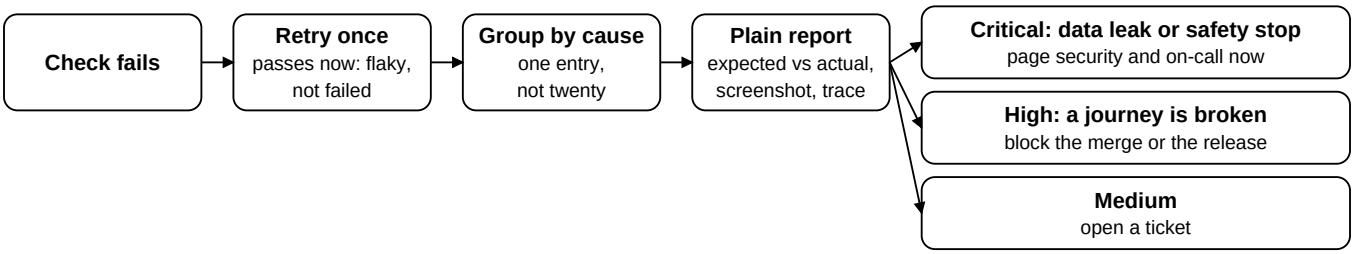

Figure 4. One clear entry per problem, routed by how serious it is.

- **No false alarms.** Site down is one result, not twenty. Live checks must fail twice before paging anyone.
- **Easy to read.** Each job ends with a short Claude summary and a recording. The summary never changes the verdict.

7. Results

Command	Result
`npm run qa`	PASS, 19 checks, 36 seconds (latest run, Nov 8).
`npm run qa:bugs`	9 of 9 planted bugs caught. A slow but working page still passes.
`npm run qa:repeat`	10 runs in a row, same result every time.

Built with Claude Code as a pair programmer; every decision and result was reviewed and run locally.